Batch: 6
Topic: ProtLoc Fusion

Proposed Work

- Five separate published tools are used today to answer five related questions about the same protein sequence. A researcher runs each, in a different input format, and reconciles the answers by hand.

- ProtLoc Fusion answers all of them from one shared representation : one sequence in, one organism-group selection, one output.

- Output covers: subcellular location (multi-label), membrane type, N-terminal sorting signal + cleavage site, GO function, homology evidence, per-residue explanation.

- Delivered as a web server / API with whole-proteome batch mode : not a library, not a CLI.

Problem / gap being addressed

Tool	Covers	Limitation
DeepLoc	Eukaryote localization + membrane type	Eukaryote-only
TargetP	N-terminal presequences	Plant vs non-plant only
SignalP	Signal peptide type + cleavage	Signal peptides only
WoLF PSORT	Hand-crafted features, k-NN	Organism-group specific
CELLO / PSORTb	Prokaryotic localization	SVM-era, prokaryote-only

Technology stack

Layer	Choice	Justification
Language	Python 3.11	Only ecosystem with mature DL *and* classical ML
DL framework	PyTorch 2.x	Required by `fair-esm` and HuggingFace protein LMs
Protein LMs	ESM-2 (`esm2_t33_650M_UR50D`), ProtT5, ProtBERT	LocPro uses ESM2; PLM-effector finds protein-specific PLMs beat general LMs; DeepLoc 2.1 finds ProtT5 marginally over ESM-1B → benchmark more than one
Trunk adaptation	LoRA / PEFT	PEFT-SP: up to **87.3% MCC gain on small-sample categories**, lower compute than adapter tuning
Classical ML	scikit-learn	LogReg / SVM / RF baselines
Calibration	netcal	Temperature / vector scaling
Config	Hydra + OmegaConf, validated by Pydantic v2	Every experiment reproducible from a config file; no magic numbers in code
Tracking	Weights & Biases (offline default) + TensorBoard	Ablation bookkeeping across 16 phases
Serving	FastAPI + Uvicorn + Jinja2	Async batch endpoints for proteome-scale jobs
Storage	Filesystem, content-addressed (no RDBMS in v1)	Corpus is immutable and append-only; a DB adds ops burden with no benefit. SQLite only if job state outgrows files
Testing / QA	pytest, ruff, mypy, import-linter	Phase gates + dependency-rule enforcement
Packaging	Docker (GPU-optional)	Target is a server; a pip package is explicitly out of scope

Module design

L-1 : Domain (the foundation everything else stands on)

- **`domain`** : This is the shared vocabulary of the entire system: the data classes, enums, and (critically) the *port protocols* : the interfaces every other module implements or consumes. It imports nothing from the rest of the codebase, which is what makes it the true dependency root. Its contribution: it's the contract that lets 16 other modules stay decoupled from each other. Change a port signature here and you've changed the API for the whole system in one place.

L0 : Delivery mechanisms (how the outside world talks to the system)

- **`api`** : The HTTP-facing adapter. It parses incoming requests, validates shape (not business logic : just "is this well-formed"), and delegates to `inference` or `jobs`. It contributes the machine-readable entry point (REST/JSON) without ever containing prediction logic itself : that discipline is what keeps it swappable.
- **`web`** : The human-facing adapter. Takes a domain response object and renders it as HTML. Nothing about *how* a prediction was made lives here : purely presentation. Its contribution is the browser-based UI, decoupled enough that it could be replaced with a different frontend without touching prediction code.
- **`jobs`** : Manages whole-proteome (many-sequence) runs: splitting a huge input into chunks, tracking progress, handling cancellation. This is the layer that makes ProtLoc Fusion usable at scale instead of one-sequence-at-a-time : its contribution is turning "run this for 20,000 proteins" into a manageable, resumable, cancellable operation.

L1 : Orchestration and configuration

- **`inference`** : The conductor. This is the only place where the order of the prediction pipeline (features → model → calibration → consistency → interpretability → evidence) is written down. Every other module executes a step; this module decides the sequence. Its contribution is that changing the pipeline order is a one-file change, not a hunt across the codebase.
- **`config`** : The single source of truth for every hyperparameter, file path, and model setting. Nothing else is allowed to hardcode a magic number or path : it all flows from here. Contribution: reproducibility and one place to tune the whole system for an experiment or deployment.

L2 : Feature extraction

- **`features`** : Converts raw sequences into tensors. Each feature family (protein language model embeddings, classical hand-crafted features, k-mers, secondary structure) gets its own adapter, but all of them sit behind one shared port so `inference` never needs to know which specific feature extractor is in play. Contribution: this is where DeepLoc-style embeddings, PSORT-style motifs, and everything else the scope calls for actually get computed.

L3 : Models and training

- **`models`** : Houses the shared trunk (the common backbone), the 7 task-specific prediction heads (eukaryotic localization, membrane type, prokaryotic Gram +/-, signal peptide, mitochondrial transit peptide, chloroplast transit peptide, multi-label), and the classical baselines (LogReg/SVM/RF). The rule that heads never import each other is what enforces true multi-task independence : one head's bug can't silently leak into another's output. Contribution: this is the predictive core of the project, and the one module most directly answerable to "how good are the predictions."
- **`training`** : Fits models, either single-task or jointly across tasks. It's explicitly *not* used at serving/inference time : training and serving are different concerns with different lifecycles, so this module can be heavy, slow, and GPU-hungry without that cost ever touching the live prediction path.

L4 : Turning raw scores into trustworthy answers

- **`calibration`** : Converts raw model scores into calibrated probabilities and makes the abstention decision (i.e., "should the model actually answer, or say 'not confident enough'?"). This directly serves the nice-to-have of confidence-calibrated/uncertainty-aware outputs from your scope.
- **`consistency`** : Checks whether a *joint* prediction across multiple tasks is biologically coherent (e.g., a protein predicted as "secreted" shouldn't also be flagged "no signal peptide"). The rules live in YAML, not code : meaning a biologist can edit constraints without a code change or redeploy. Contribution: this is a big differentiator over the standalone legacy tools, since none of DeepLoc/PSORT/TargetP/SignalP/CELLO individually cross-checks a *combined* multi-label answer for internal contradiction.

L5 : Explainability and external corroboration

- **`interpretability`** : Produces per-residue attribution scores derived from the trunk's attention weights : i.e., "which parts of the sequence drove this prediction." This is one of your locked core-scope items (attention/interpretability visualization), and its contribution is trust and debuggability: a researcher can see *why*, not just *what*.
- **`evidence`** : Surfaces external corroboration for a prediction: nearest-neighbor homologs and GO term annotations. This gives a prediction outside support beyond the model's own confidence : directly matching your core-scope items for homology evidence display and GO functional annotation.

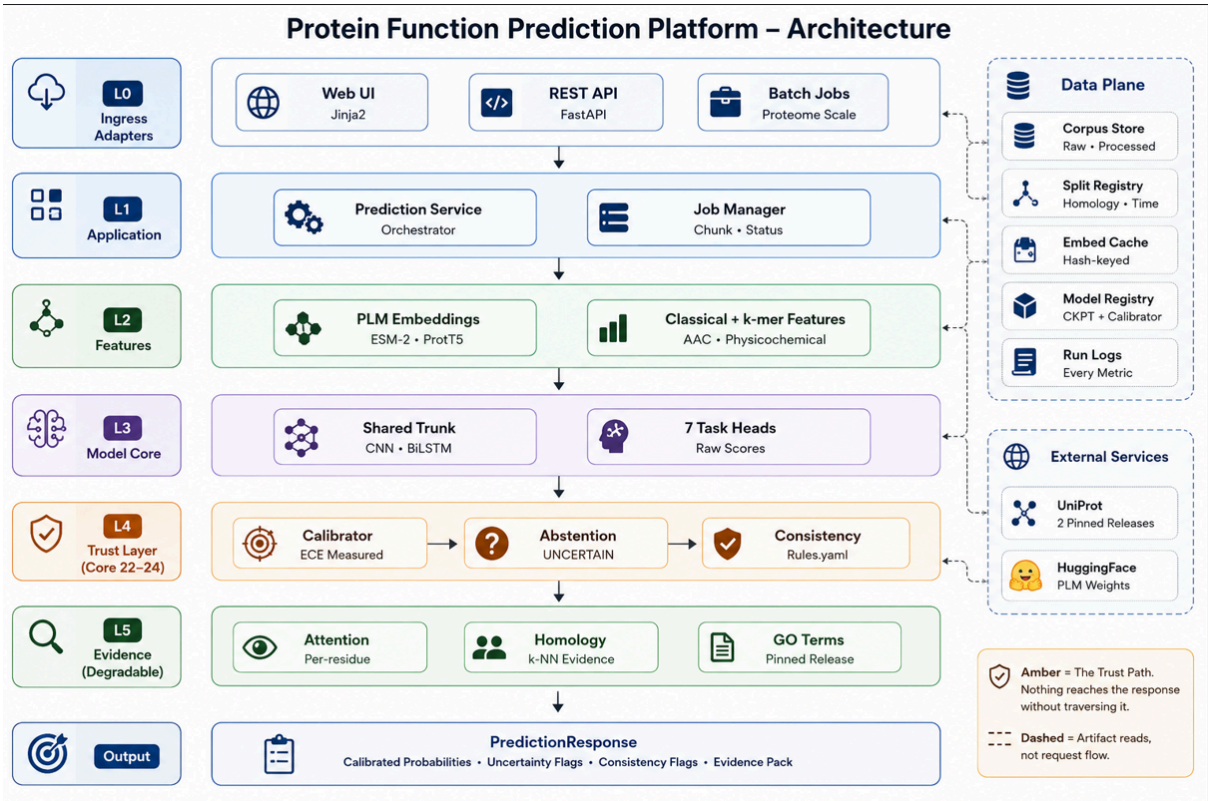
Data-layer modules (cross-cutting, not part of the live request path)

- **`data`** : Handles corpus I/O and the three split families (presumably train/val/test-style splits, possibly by organism/homology/task), plus leakage assertions : automated checks that no test-set sequence has leaked into training via homology or duplication. Contribution: this is what makes benchmark numbers actually trustworthy rather than optimistic.
- **`registry`** : Bundles a trained checkpoint, its calibrator, and its consistency rules into one atomic, versioned unit. This guarantees you never accidentally serve a model with a mismatched (stale or incompatible) calibrator or rule set : a subtle bug class that's easy to hit without this discipline.
- **`evaluation`** : Runs metrics and benchmark harnesses, but pointedly calls the *served* path (through the real API/inference stack) rather than calling `models` directly. Contribution: your benchmark numbers reflect exactly what a user would actually get, not an idealized shortcut through the internals.

Cross-cutting utility

- **`utils`** : Logging, seeding, hashing. Contains zero business logic by design : it's infrastructure plumbing every other module can depend on without violating the layering rule, since it sits outside the domain-flow direction entirely.

ARCHITECTURE DIAGRAM



,